

TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA: CÔNG NGHỆ THÔNG TIN
BỘ MÔN: CÔNG NGHỆ PHẦN MỀM

ĐỀ THI GIỮA KỲ

Tên học phần: Toán ứng dụng CNTT

Mã học phần:

Số tín chỉ: **03**

Phương pháp đánh giá (*): Tự luận có giám sát

Thời gian làm bài: **90** phút

Đề số: **T000.01**

☐ Sinh viên được sử dụng tài liệu khi làm bài.

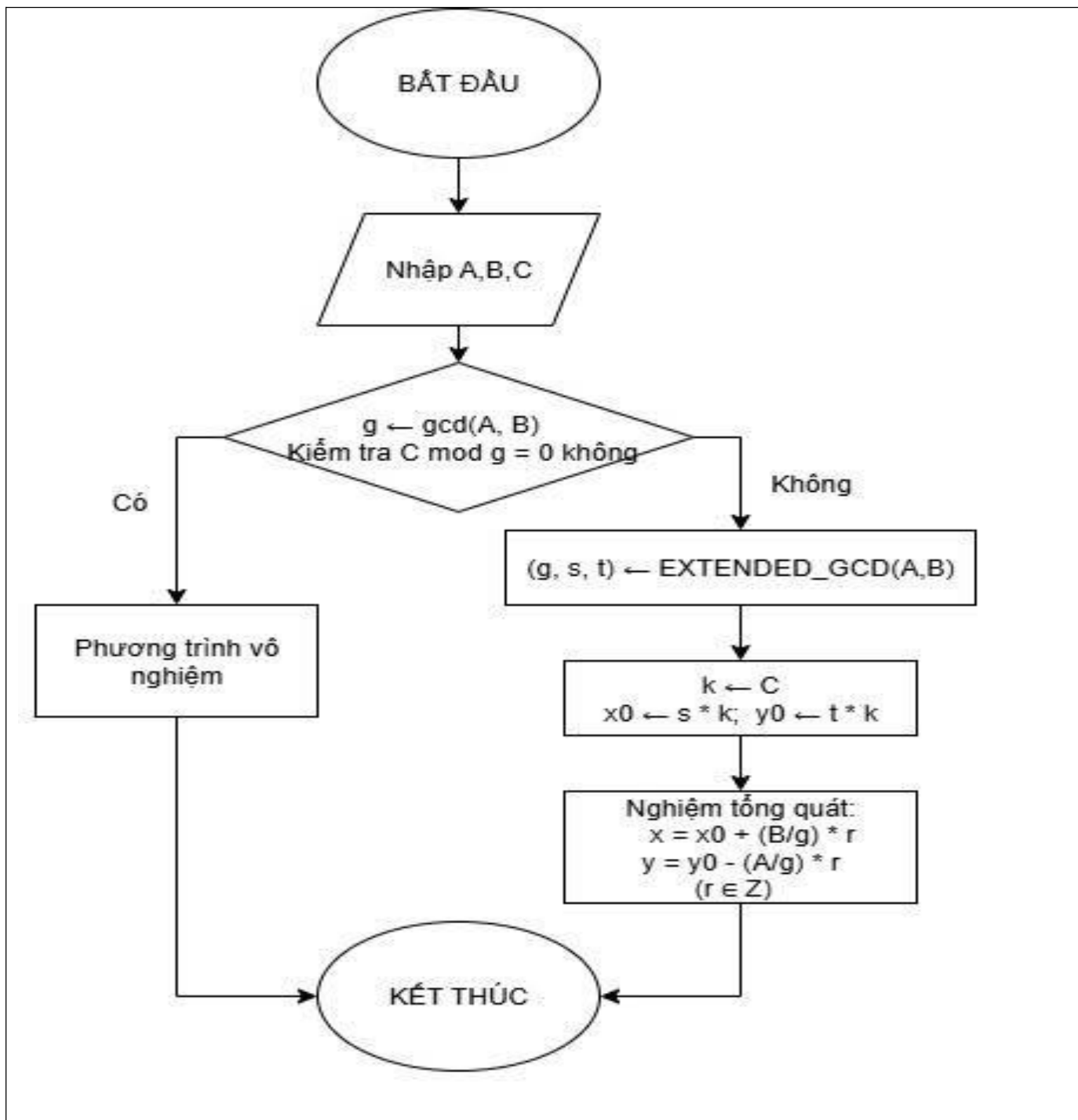
Họ tên: Lê Duy Việt **Lớp:**24T_DT2 **MSSV:**102240121

Sinh viên làm bài trực tiếp trên tệp này, lưu tệp với định dạng MSSV_HọTên.pdf và nộp bài thông qua MSTeam

Câu 1 (3 điểm):

- a) (1 điểm) Cho phương trình $Ax + By = C$ với A, B, C là các số nguyên dương và x, y là biến. Hãy trình thuật toán để giải phương trình trên.

Trả lời: Trình bày thuật toán bằng sơ đồ khối



b) (1 điểm) Hãy viết chương trình sử dụng thuật toán để giải phương trình $Ax + By = C$

Trả lời: Dán code vào bên dưới

```

//phương trình nghiệm nguyên
#include <iostream>
#include<math.h>
using namespace std;
int ExtendedGcd(int a, int b, int &x, int &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = ExtendedGcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a/b)*y1;
}

```

```

    return d;
}
void Diophantine(int a, int b, int c) {
    int x, y;
    int d = ExtendedGcd(a, b, x, y);
    if (c % d != 0) {
        cout << "Vo nghiem.\n";
        return;
    }
    x *= c / d;
    y *= c / d;
    cout << "Nghiem rieng: x = " << x << ", y = " << y << endl;
    cout << "Nghiem tong quat:" << endl;
    cout << "x = " << x << " + " << b/d << "*r " << endl;
    cout << "y = " << y << " - " << a/d << "*r ";
}
int main() {
    int a, b, c;
    cout << "Nhap a, b, c: ";
    cin >> a >> b >> c;
    Diophantine(a, b, c);
    return 0;
}

```

c) (1 điểm) Hãy minh họa kết quả chương trình

Trả lời: Dán kết quả gồm nghiệm riêng và nghiệm tổng quát của phương trình $3x + 9y = 27$.

```

Nhap a, b, c: 3 9 27
Nghiem rieng: x = 9, y = 0
Nghiem tong quat:
x = 9 + 3*r
y = 0 - 1*r

```

Câu 2 (4 điểm): Cho ma trận A

a) (1 điểm) Trình bày thuật toán để phân rã ma trận A theo SVD

Trả lời: Trình bày thuật toán bằng sơ đồ khối hoặc ngôn ngữ tự nhiên ở đây:

1. Tính $A^T \cdot A$.
2. Tìm giá trị riêng và vector riêng của $A^T \cdot A$.
3. Các vector riêng(λ) tạo thành ma trận V.
4. Các căn bậc hai của giá trị riêng là các giá trị kỳ dị ($\sigma = \sqrt{\lambda}$), tạo thành Σ .
5. Tính $U = A \cdot V / \sigma(i)$.
6. Bổ sung trục chuẩn để U đầy đủ

Kết quả: $A = U \cdot \Sigma \cdot V^T$.

b) (1 điểm) Viết chương trình để thực hiện phân rã SVD

Trả lời: Dán code vào bên dưới:

```
#include <iostream>
#include <vector>
#include <math.h>
#include <iomanip>
#include <algorithm>
using namespace std;

#define EPS 1e-9
#define MAX_ITER 500

vector<vector<double>> nhanmatran(const vector<vector<double>>& A, const vector<vector<double>>& B) {
    int n = A.size(), m = B[0].size(), p = B.size();
    vector<vector<double>> C(n, vector<double>(m, 0));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            for (int k = 0; k < p; k++)
                C[i][j] += A[i][k] * B[k][j];
    return C;
}

vector<vector<double>> chuyenvi(const vector<vector<double>>& A) {
    int n = A.size(), m = A[0].size();
    vector<vector<double>> B(m, vector<double>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            B[j][i] = A[i][j];
    return B;
}

void chuanhoa(vector<double>& v) {
    double norm = 0;
    for (double x : v) norm += x * x;
    norm = sqrt(norm);
    if (norm < EPS) return;
    for (double &x : v) x /= norm;
}

pair<double, vector<double>> tririengpoweriteration(const vector<vector<double>>& A) {
    int n = A.size();
    vector<double> v(n, 1.0);
    chuanhoa(v);
    double lambda = 0;
    for (int iter = 0; iter < MAX_ITER; iter++) {
        vector<double> Av(n, 0);
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                Av[i] += A[i][j] * v[j];

        double newlambda = 0;
        for (int i = 0; i < n; i++) newlambda += v[i] * Av[i];

        if (fabs(newlambda - lambda) < EPS) break;
        lambda = newlambda;
        v = Av;
        chuanhoa(v);
    }
    return make_pair(lambda, v);
}
```

```
void deflation(vector<vector<double>>& A, double Lambda, const vector<double>& v) {
    int n = A.size();
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            A[i][j] -= Lambda * v[i] * v[j];
}
```

```
void phanraSVD(const vector<vector<double>>& A) {
    int m = A.size(), n = A[0].size();
    vector<vector<double>> AT = chuyenvi(A);
    vector<vector<double>> ATA = nhanmatran(AT, A);
    vector<pair<double, vector<double>>> eigens;
    vector<vector<double>> B = ATA;
    for (int i = 0; i < n; i++) {
        auto result = tririengpoweriteration(B);
        double lambda = result.first;
        vector<double> v = result.second;
        eigens.push_back({lambda, v});
        deflation(B, lambda, v);
    }
}
```

```
// Sắp xếp theo trị riêng giảm dần
sort(eigens.begin(), eigens.end(), [](auto &a, auto &b) {
    return a.first > b.first;
});
```

```
// V
vector<vector<double>> V(n, vector<double>(n));
vector<double> sigma;
for (int i = 0; i < n; i++) {
    sigma.push_back(sqrt(max(eigens[i].first, 0.0)));
    for (int j = 0; j < n; j++)
        V[j][i] = eigens[i].second[j];
}
// U
vector<vector<double>> U(m, vector<double>(m, 0));
for (int i = 0; i < n; i++) {
    vector<double> Av(m, 0);
    for (int j = 0; j < m; j++)
        for (int k = 0; k < n; k++)
            Av[j] += A[j][k] * V[k][i];
    if (sigma[i] > EPS) {
        for (int j = 0; j < m; j++)
            U[j][i] = Av[j] / sigma[i];
    }
}
}
```

```
// Bổ sung vector trục chuẩn thiếu trong U
for (int k = n; k < m; k++) {
    vector<double> u(m, 0);
    u[k] = 1.0;
    for (int j = 0; j < k; j++) {
        double dot = 0;
        for (int i = 0; i < m; i++) dot += U[i][j] * u[i];
        for (int i = 0; i < m; i++) u[i] -= dot * U[i][j];
    }
    chuanhoa(u);
    for (int i = 0; i < m; i++) U[i][k] = u[i];
}
cout << "\nMa tran U:\n";
for (int i = 0; i < m; i++) {
    for (int j = 0; j < m; j++)
        cout << setw(10) << fixed << setprecision(2) << U[i][j] << " ";
    cout << "\n";
}
```

```

}
cout << "\nGia tri ky di (Sigma):\n";
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        if (i == j && j < sigma.size())
            cout << setw(10) << fixed << setprecision(2) << sigma[j];
        else
            cout << setw(10) << 0.0;
    }
    cout << "\n";
}
}

```

```

cout << "\nMa tran V:\n";
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++)
        cout << setw(10) << fixed << setprecision(2) << V[i][j] << " ";
    cout << "\n";
}
}
int main() {
    int m, n;
    cout << "Nhap kích thước ma tran m n: ";
    cin >> m >> n;
}

```

```

vector<vector<double>> A(m, vector<double>(n));
cout << "Nhap ma tran A:\n";
for (int i = 0; i < m; i++)
    for (int j = 0; j < n; j++)
        cin >> A[i][j];
}

```

```

phanraSVD(A);
return 0;
}

```

c) (1 điểm) Trình bày kết quả phân rã SVD

Trả lời: Dán kết quả minh họa với ma trận

1	1	2	3
2	0	1	-4
3	1	3	-1

Nhap kích thước ma trận m n: 3 4

Nhap ma trận A:

1 1 2 3
2 0 1 -4
3 1 3 -1

Ma trận U:

-0.06 0.81 0.00
0.74 -0.35 0.00
0.67 0.46 0.00

Gia trị kỳ dị (Sigma):

5.79 0.00 0.00 0.00
0.00 4.74 0.00 0.00
0.00 0.00 0.00 0.00

Ma trận V:

0.59 0.32 0.50 0.50
0.11 0.27 0.50 0.50
0.45 0.56 0.50 0.50
-0.66 0.72 0.50 0.50

d) (1 điểm) Nêu 1 ứng dụng về ứng dụng SVD và em hiểu biết sâu sắc nhất

Trả lời: Dán nội dung

Collaborative Filter: Gợi Ý

Giả sử có bảng rating:

You	Film A	Film B	Film C
U1	5	?	3
U2	4	2	?
U3	?	3	4

Dấu '?' là rating chưa biết (You chưa xem).

Dùng SVD ta xấp xỉ bảng này bằng công thức:

$$R \approx U(k) \Sigma(k) V(k)^T$$

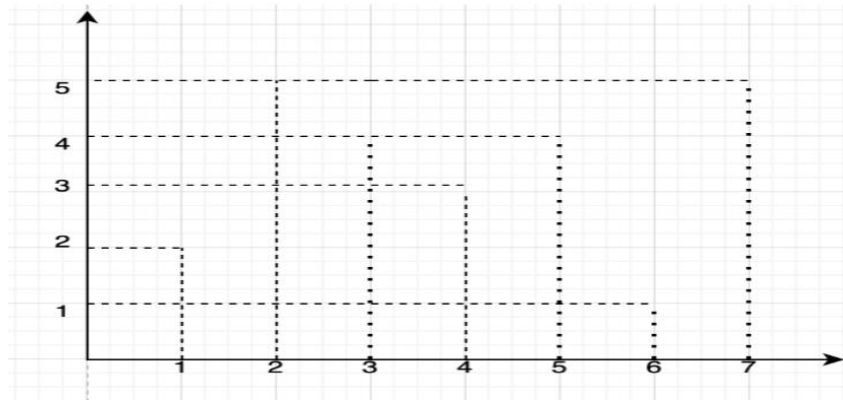
- $U(k)$: biểu diễn người dùng trong không gian đặc trưng ẩn (ví dụ: thích hành động, thích lãng mạn).
- $V(k)$: biểu diễn phim trong cùng không gian đặc trưng.
- $\Sigma(k)$: mức độ quan trọng của từng đặc trưng.

Dự đoán rating chưa biết:

$$R(ij) = (U(k) \Sigma(k) V(k)^T)(ij)$$

Ví dụ: hệ thống có thể dự đoán U1 sẽ cho Film B = 4.2, từ đó gợi ý U1 nên xem Film B.

Câu 3 (3 điểm): Cho không gian Oxy và 7 điểm tương ứng như hình vẽ dưới:



a) (1 điểm) Trình bày thuật toán xác định bao lồi

Trả lời: dán sơ đồ khối hoặc ngôn ngữ tự nhiên vào bên dưới:

- Sắp xếp điểm**: Sắp xếp các điểm theo tọa độ (x) tăng dần (nếu (x) bằng nhau thì theo (y)).
- Xây bao dưới**:
 - Duyệt từ điểm trái nhất, thêm điểm mới nếu tạo "rẽ trái" (tích có hướng > 0).
 - Nếu tạo "rẽ phải" hoặc thẳng hàng (tích ≤ 0), loại bỏ điểm cuối và kiểm tra lại.
- Xây bao trên**:
 - Duyệt ngược từ điểm phải nhất, thêm điểm mới nếu tạo "rẽ trái" (tích có hướng > 0).
 - Nếu tạo "rẽ phải" hoặc thẳng hàng (tích ≤ 0), loại bỏ điểm cuối và kiểm tra lại.
- Kết hợp**: Nối bao dưới và bao trên, xóa hai điểm trùng ở đầu/cuối.
- Kết quả**: Danh sách đỉnh bao lồi theo thứ tự ngược chiều kim đồng hồ.

b) (1.5 điểm) viết chương trình xác định bao lồi và tính diện tích bao lồi

Trả lời: viết câu trả lời vào bên dưới:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>
using namespace std;

struct Diem {
    int x, y;

    bool operator<(const Diem& diem_khac) const {
        return (x < diem_khac.x) || (x == diem_khac.x && y < diem_khac.y);
    }
};

int tichcohuong(const Diem& goc, const Diem& a, const Diem& b) {
    return (a.x - goc.x) * (b.y - goc.y) - (a.y - goc.y) * (b.x - goc.x);
}

vector<Diem> timbaoloi(vector<Diem>& tap_diem) {
    int n = tap_diem.size();
    if (n <= 1) return tap_diem;
    sort(tap_diem.begin(), tap_diem.end());
    vector<Diem> bao_duoi, bao_tren;
    // Dưới
```

```

        for (const auto& diem : tap_diem) {
            while (bao_duoi.size() >= 2 && tichcohuong(bao_duoi[bao_duoi.size()-2], bao_duoi[bao_duoi.size()-1],
diem) <= 0) {
                bao_duoi.pop_back();
            }
            bao_duoi.push_back(diem);
        }
        // Trên
        for (int i = n - 1; i >= 0; i--) {
            const Diem& diem = tap_diem[i];
            while (bao_tren.size() >= 2 && tichcohuong(bao_tren[bao_tren.size()-2], bao_tren[bao_tren.size()-1],
diem) <= 0) {
                bao_tren.pop_back();
            }
            bao_tren.push_back(diem);
        }
        // Xóa trùng
        bao_duoi.pop_back();
        bao_tren.pop_back();
        // Ghép 2 bao lại
        bao_duoi.insert(bao_duoi.end(), bao_tren.begin(), bao_tren.end());
        return bao_duoi;
    }
}

```

```

void in(const vector<Diem>& bao_Loi) {
    cout << "\nCac diem thuoc bao loi theo thu tu:\n";
    for (const auto& diem : bao_Loi) {
        cout << diem.x << " " << diem.y << "\n";
    }
}

```

```

double tinhdientich(const vector<Diem>& bao_Loi) {
    double S = 0;
    int n = bao_Loi.size();
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        S += (bao_Loi[i].x * bao_Loi[j].y - bao_Loi[j].x * bao_Loi[i].y);
    }
    return fabs(S) / 2.0;
}

```

```

int main() {
    int n;
    cout << "Nhap so luong diem: ";
    cin >> n;
    vector<Diem> tap_diem(n);
    cout << "Nhap toa do cac diem (x y):\n";
    for (int i = 0; i < n; ++i) {
        cin >> tap_diem[i].x >> tap_diem[i].y;
    }
    vector<Diem> bao_loi = timbaoloi(tap_diem);
    in(bao_loi);
    double dientich = tinhdientich(bao_loi);
    cout << "\nDien tich bao loi: " << dientich << endl;
}

```

```

    return 0;
}

```

c) (0.5 điểm) Chỉ ra hai điểm trên bao lồi có kết khoảng cách ngắn nhất

Trả lời: viết câu trả lời vào bên dưới:

Khi ta xác định được các điểm trên bao lồi thì ta tính khoảng cách của từng cặp 2 điểm trên bao lồi và tìm được 2 điểm gần nhất trên bao lồi và khoảng cách của nó

Hai điểm gần nhau nhất trên bao lồi là: (1,2) và (2,5)

Khoảng cách đó là: $\sqrt{10}$

GIẢNG VIÊN BIÊN SOẠN ĐỀ THI

Đà Nẵng, ngày 18 tháng 09 năm 2025
TRƯỞNG BỘ MÔN
(đã duyệt)